

Adversarial Backtest Verification: Catching Overfitting, Beta, Collinearity, and Look-Ahead in a Live Trading-Research Program

A practical, example-driven protocol for avoiding false discovery in quantitative strategy research

Aleksandr Dashyan

Independent Researcher, Yerevan, Armenia

Correspondence: alikdashyan1@gmail.com

Working paper. Preliminary. Comments welcome. This version: August 2026.

JEL classification: C52, C53, G14, G17, C12

Keywords: backtest overfitting, data snooping, multiple testing, family-wise error, permutation tests, probability of backtest overfitting, market-neutral benchmarking, reproducible research, negative results

Copyright 2026 Aleksandr Dashyan. All rights reserved. This paper is circulated for discussion and comment only; it may not be reproduced, distributed, quoted at length, or used, in whole or in part, without the author's prior written permission.

Abstract

Trading research has one expensive failure mode: a strategy that backtests well, gets funded, and then loses money, while the opportunities you correctly pass on cost you nothing. This paper gathers the checks I leaned on to avoid that failure over several months of work on cryptocurrency perpetual futures, and sets them down as a protocol for adversarial verification. There are seven safeguards, each paired with a moment in the program where skipping it would have changed the answer. Two of those moments get worked through in full, because both looked like discoveries and neither held up: an account return that hung on an unstable tie-break, and a directional result no permutation test could separate from chance. The safeguards themselves are to hold out time and expect the in-sample edge to evaporate; to benchmark against the one-sided baseline that beta hands you for free rather than against fifty percent; to collapse correlated events before computing significance; to deflate for the number of rules searched; to guard the labels and the clock against look-ahead; to defend the raw signal instead of the equity curve; and to pin down whatever in the implementation is not supposed to matter. None of the individual checks is new, and integrated backtesting protocols already exist (Arnott,

Harvey and Markowitz, 2019; Lopez de Prado, 2018). The contribution is a compact worked checklist, each item tied to a specific live failure with the cost of skipping it named, applied by the researcher against the result he most wanted to keep. Four computations on the program's own data anchor the claims. A deflated Sharpe ratio pulls the large in-sample Sharpe below the level that searching over so many rules produces from pure noise. A purge-and-embargo re-split leaves the out-of-sample null almost exactly where it was. A stationary block bootstrap does the work on the equity example, and a positive control confirms that the central permutation test has the right size and real power. Run end to end, the protocol turned two apparent discoveries into one confirmed artifact and one result indistinguishable from chance, and what it left was a null.

1. Introduction

Whoever runs a backtest chooses what to measure, and measures it on data they have already seen. The costs of getting it wrong are lopsided. A false negative just forgoes a trade that was never taken; a false positive is the dangerous one, sized up with leverage until it loses real capital. That asymmetry is the trading version of a more general point about inference when true effects are rare (Ioannidis, 2005): when most candidate edges are null, most of the ones that look positive are false, so the bar for believing any of them has to be set high. Several months of running such a program taught me the shape of the failure by repetition. A rule, a feature set, or a model would post a large in-sample edge and then fail out of sample, and the few that cleared a first out-of-sample pass tended to fall apart on a second look. The causes varied: sometimes beta, sometimes correlated events padding the sample, sometimes an artifact of how the portfolio was assembled or an implementation detail nobody expected to matter. What follows is an account of the checks that caught those failures, and of what each one was worth.

The data-snooping literature is mature, and integrated backtesting protocols already exist (Arnott, Harvey and Markowitz, 2019; Lopez de Prado, 2018). Rarer is a worked demonstration, on live data, of each failure mode actually occurring, being caught, and changing the conclusion, alongside an honest post-mortem of two positive results that did not survive and a number attached to the cost of skipping each check. The methods themselves are standard. What earns its keep, I think, is watching them turned adversarially on the researcher's own preferred answer, and seeing what each one buys. The seven safeguards occupy Sections 4 through 10, one at a time with an example each; the two case studies sit side by side in Section 11 and the protocol

condenses to a checklist in Section 12, with background in Section 2, the setting in Section 3, and discussion and conclusion in Sections 13 and 14.

2. Background

The building blocks are well known. Lo and MacKinlay (1990) documented data-snooping biases in tests of asset-pricing models. White (2000) gave a reality check for picking the best of many rules, and Sullivan, Timmermann and White (1999) ran exactly that check over a large universe of technical trading rules with a bootstrap correction, which is the closest prior setting to the rule search here. Hansen (2005) fixed the reality check's conservatism with a test for superior predictive ability; Romano and Wolf (2005) went further, with a stepwise procedure that controls family-wise error and says which rules survive, not just whether the best one does. Harvey and Liu (2015) and Harvey, Liu and Zhu (2016) measured how badly multiple testing inflates apparent significance, in backtesting and across the factor zoo, and argued for thresholds well above the usual two sigma. On the Sharpe side, Bailey and Lopez de Prado (2014) introduced the deflated Sharpe ratio, Bailey, Borwein, Lopez de Prado and Zhu (2014) showed how few trials it takes to conjure a spurious in-sample optimum, and the same authors (2017) turned that into the probability of backtest overfitting through combinatorially symmetric cross-validation. Lopez de Prado (2018) set out purged and embargoed cross-validation for financial time series, where ordinary cross-validation leaks. Aronson (2007) made the general case for evidence-based testing of technical rules with explicit multiple-testing correction.

Those protocols already exist, then. Arnott, Harvey and Markowitz (2019) give a backtesting protocol for the machine-learning era; Lopez de Prado (2018) offers checklists in the same spirit. I am not trying to extend the theory or to displace them. This paper does something narrower. It turns the theory into a short run of adversarial checks and shows, on one program's live data, what each check catches, what skipping it costs, and how it behaves when it is aimed at a result the author was hoping to keep. The existing protocols are general and prescriptive; this one is diagnostic and worked, every item earned by a specific failure in a running system.

3. The Setting

The examples come from a research program with two aims: to predict the direction of the next short-horizon move in liquid cryptocurrency perpetual futures from a large multi-source

signature, and, on a separate track, to hunt for event-based edges across asset classes. The directional findings have their own paper (Dashyan, 2026). Here those same experiments serve as a supply of concrete verification failures and fixes, and I restate every number the argument needs so this paper can be read on its own.

The main dataset runs to about forty thousand resolved market snapshots, 40,857 to be exact, over twenty Binance USD-margined perpetual contracts, from roughly 15 May to 6 August 2026. A snapshot is a point-in-time reading of many features grouped into seven thematic lanes, among them funding, positioning, order flow, and liquidations, drawn from about 160 underlying sources. Each feature is encoded to a small ordinal code, and the codes within a lane are folded by majority vote into a lane state. The tuple of seven lane states is the signature; each distinct signature value is a cell, a recurring market state pooled across coins. The features are encoded at two resolutions. A three-level signature keeps only the sign of each lane, up, flat, or down, giving 3 to the 7th power possible cells. A five-level signature also records conviction, strong or weak, giving 5 to the 7th power possible cells, so its states are rarer and cut more finely. Three rules then read off the cells. A per-cell majority-winner predicts, in each cell, whichever side won most often in training. An eight-rule vote combines eight fixed, always-directional sub-rules. A best-of-combination search reports, per cell, the highest-scoring subset of those rules. Outcomes are labeled by fixed take-profit and stop-loss brackets on one-minute prices with a forty-eight-hour timeout, and an independent first-touch resolver decides which bracket is reached first.

The examples use two exit geometries. A symmetric bracket sets take-profit and stop-loss the same distance apart, so the directional break-even is fifty percent. An asymmetric two-to-one bracket puts the take-profit twice as far as the stop, and a single trade then breaks even at a win rate of one in three, 33.3 percent, since a win pays double what a loss costs. Account returns, wherever I quote them, come from one fixed simulation: each position risks a set fraction of equity (five percent margin at seven times leverage), a ninety-five percent capital cap limits how many positions run at once, at most one position per coin is open at a time, a flat 0.1 percent round-trip fee is charged, and equity compounds. Training is the data before a fixed calendar split at 1 July 2026; testing is everything from that date on. That is a single chronological hold-out rather than a rolling walk-forward, a distinction Section 4 returns to. Because the label horizon runs forty-eight hours, training snapshots within two days of the split resolve partly on

prices from after it, and Section 8 handles that overlap. Recomputing every resolved outcome independently across both bracket geometries comes to 64,434 checks, the figure the resolver cross-check in Section 8 refers to.

4. Safeguard One: Hold Out Time, and Expect the In-Sample Edge to Vanish

The first safeguard, and the one that matters most, is a strictly chronological train-test split: fix every parameter, threshold, rule polarity, and model choice on the training period, and score only on the held-out future. It is not the same as random cross-validation, which leaks in a time series, because neighboring observations are correlated and future rows end up informing past folds.

The example is blunt. On the three-level signature, a per-cell majority-winner posted a 56.0 percent directional win rate and a compounded 60.4 percent in training, then fell to 47.7 percent and minus 39.4 percent out of sample, a coin flip that loses money. Dress the same rule up with an eight-way vote and the in-sample win rate climbs to 71.7 percent while the out-of-sample rate sits at 50.1 percent, so the whole enrichment was overfit. Figure 1 shows the collapse repeating across all four methods.

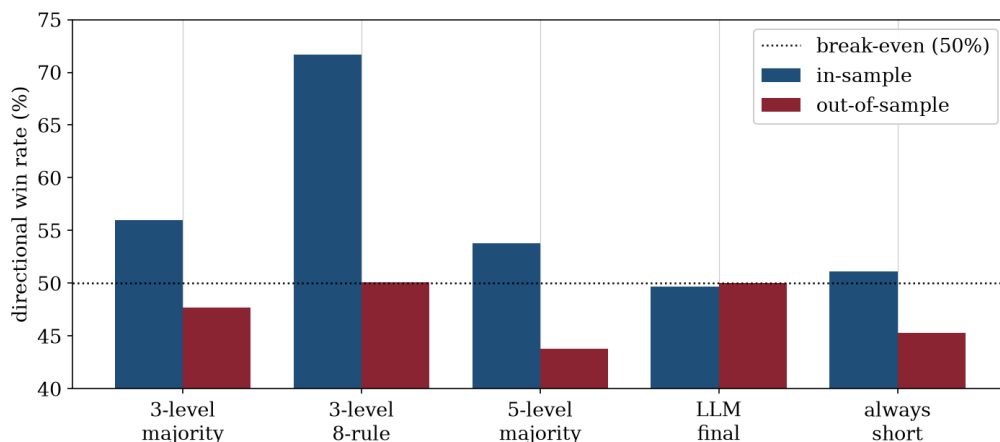


Figure 1. In-sample directional win rate (left bar of each pair) against out-of-sample (right bar) for four decision methods, the three-level per-cell majority-winner, the eight-rule vote, the five-level majority-winner, and a language-model final decision, together with the always-short baseline, at the symmetric bracket where the directional break-even is fifty percent. Every in-sample edge falls to the break-even line out of sample.

A single chronological split has its own weakness. It yields one out-of-sample estimate, which is itself a single draw that can come up lucky or unlucky, and it never says how likely an in-

sample-selected configuration is to underperform out of sample. The principled generalization is the probability of backtest overfitting, estimated by combinatorially symmetric cross-validation (Bailey, Borwein, Lopez de Prado and Zhu, 2017), which repeats the split many ways and counts how often the in-sample best fails to clear the median out of sample. A second limit is harder to escape: the program's window is one mostly down-trending regime, so even a spotless hold-out says nothing about how the rule travels across regimes. So one split is the least you should trust, combinatorial resampling is better wherever the sample can carry it, and a single-regime window stays an open threat.

5. Safeguard Two: Benchmark Against Beta, Not Against Fifty Percent

In a falling tape a book that simply defaults to short wins more than half its trades for nothing, and any method that happens to lean short inherits that same beta. So a directional win rate above fifty percent is not, on its own, evidence of skill when the market was trending. The benchmark cannot be fifty percent in the abstract. It has to be the realized win rate of the naive one-sided book at the exact times and instruments the method actually traded, which I call the beta floor. An equivalent route is to resolve outcomes market-neutral, relative to the cross-section, so that a move common to everything cancels.

In this window the naive always-short book won in the low fifties in-sample and then lost badly once the market stopped falling. Several methods that cleared fifty percent did not clear the always-short floor at the same timestamps, and calling them skillful would have been a benchmarking error. A margin over the beta floor is only a point estimate, though, with no sampling distribution of its own, so on its own it cannot be called significant. What gives it standing is the side-only permutation of Safeguard 6, which holds the count of long and short trades fixed and so holds the beta exposure fixed. Safeguard 2 and Safeguard 6 are one idea at two levels: measure skill as the margin over the one-sided baseline, then test that margin against a beta-matched permutation.

6. Safeguard Three: Correct for Correlated Events Before Computing Significance

A t-statistic assumes independent observations, and event studies break that assumption routinely, because one market event shows up on many correlated instruments and on adjacent days. Count those as independent and you inflate both the effective sample size and the

significance. This example is from the program's separate cross-asset event line rather than the crypto snapshots, so the jump to equities is deliberate.

The program looked at a classic short-term reversal, the one-day bounce after a large down move (Jegadeesh, 1990; Lehmann, 1990), on equity indices back to 1990. Taken at face value, a five percent down day gave 123 events across four indices, and the bounce was significant at a t of 4.5. The four indices are nearly collinear, though. A five percent down day on one is usually a five percent down day on all of them, so those 123 events are really only 58 distinct market days. Recompute on the distinct days and the t drops to 2.1; the three percent cut drops from 3.6 to 2.0. The fall is larger than a bare degrees-of-freedom rescaling would give. The square root of 58 over 123, times 4.5, is about 3.1, not 2.1; the honest 2.1 comes from recomputing on day-level observations, which also averages the effect across the collinear indices rather than merely shrinking the sample. Collapsing to distinct days removes the cross-sectional duplication but leaves the serial dependence, because extreme down days themselves cluster in volatility regimes like 2008 and 2020. The fuller fixes keep every observation and adjust the variance, through cluster-robust or heteroskedasticity-and-autocorrelation-consistent standard errors, or a stationary block bootstrap that resamples blocks of days (Politis and Romano, 1994). Run on the distinct crash dates, where it also absorbs that clustering, the block bootstrap leaves the three percent effect a hair under the conventional threshold (t about 1.9, one-sided bootstrap probability about 0.03), which confirms the modest reading. The five percent cut, with only 58 distinct crash days, is too thin to support a stable serial-robust estimate, a reminder in itself of how small the independent sample is. So the effect is real but modest, and the naive version overstated the evidence by roughly a factor of two in the t . Reduce to independent events before any significance test, and where dependence remains, prefer a variance estimator that accounts for it over a bare count of distinct events. The same trap has a cross-sectional twin in the crypto case of Section 11, where a single coin supplies most of the apparent profit.

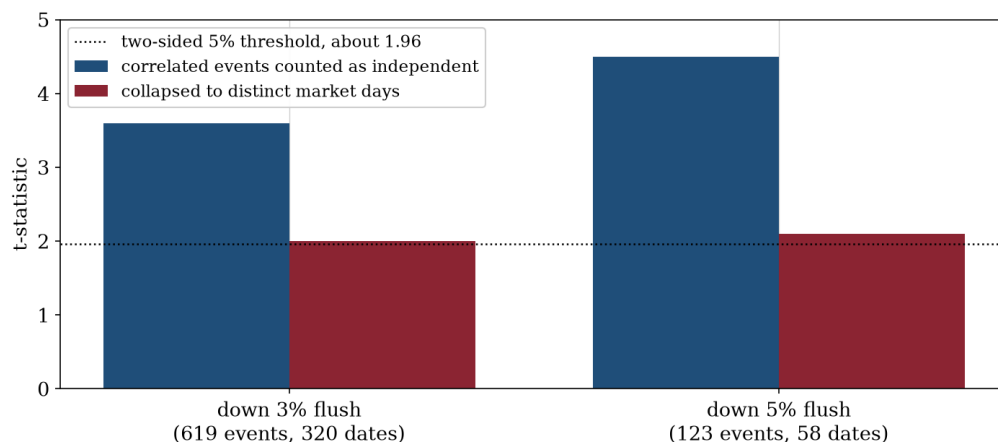


Figure 2. The same equity one-day reversal effect after a five percent and a three percent down day, with correlated events counted as independent (left bar of each pair) and honestly collapsed to distinct market days (right bar). The honest t-statistics, 2.1 and 2.0, are about half the naive 4.5 and 3.6 and sit near the conventional threshold of about 2.

7. Safeguard Four: Deflate for the Number of Rules Searched

If you screen many candidate rules or states and report the best one, it will look good even when none of them carries any signal. For K independent noise draws the expected largest absolute t grows roughly like the square root of twice the natural log of K , the curve Figure 3 draws. Two things qualify that curve. The square root of two $\ln K$ is only the leading term of the expected maximum; the finite-sample expectation runs a little lower (near K of two thousand it is closer to 3.5 than 3.9), and the maximum itself has a spread of about 0.3, so the honest comparison is against a distribution rather than a single point. And the formula assumes independent draws, which the program's candidates are not: the roughly 1,800 populated three-level cells are built from overlapping data and nested lane votes, so the effective number of independent tests, and with it the real bar, is below the nominal count. What K counts matters as well. Take all populated three-level cells and K is on the order of 1,800 (against a 3 to the 7th power ceiling of 2,187), with a naive independent bar around 3.9; take only the cells carried into the out-of-sample test, those with at least forty in-sample snapshots, and K falls to about 289, with a bar around 3.4. Either way the conclusion holds, and it is the whole point of the safeguard: a screened result reported at a t of 3 is already at or below what a search this large throws off from noise alone.

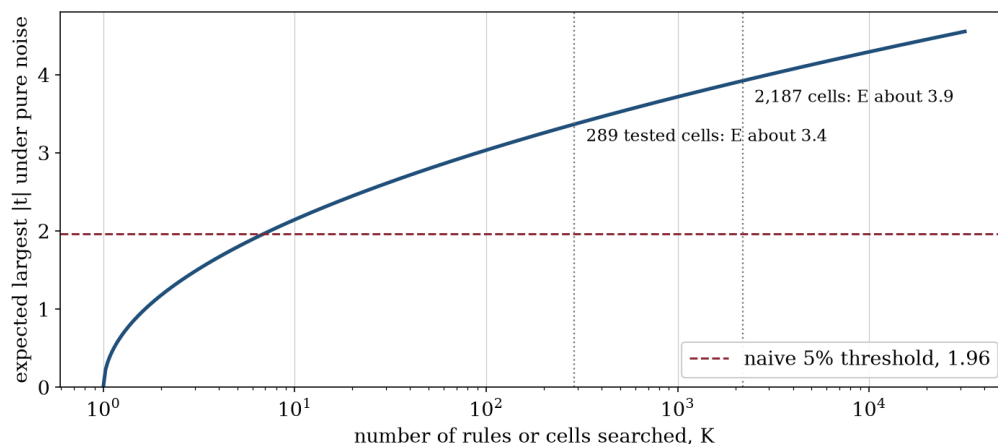


Figure 3. Leading-order expected largest absolute t -statistic under pure noise, the square root of two times the natural logarithm of K , as a function of the number of independent candidates K searched. At the program's search sizes, hundreds of rules and thousands of cells, a t near three is close to the noise expectation rather than evidence; the true finite-sample expectation is somewhat below this leading term and the maximum has non-trivial spread.

Correcting for the search is a solved problem with named tools, and the loose instruction to clear a deflated bar is really a pointer to them. White (2000) gives the reality check for the best of many rules; Sullivan, Timmermann and White (1999) apply it to a technical-rule search with a bootstrap; Hansen (2005) removes the reality check's conservatism; Romano and Wolf (2005) give a stepwise family-wise-error procedure that names which rules survive, not only whether the best one does. All of these handle the cross-rule dependence the independent-draw heuristic ignores, and they are the right tools when a search combines many signals, where picking and recombining the best inflates performance beyond what single-signal corrections catch (Novy-Marx, 2016). The deflated Sharpe ratio (Bailey and Lopez de Prado, 2014) is a separate tool, and one often confused with those: it corrects a Sharpe's own sampling distribution for the number of trials, for non-normality, and for track-record length. That makes it the right deflation when the headline is a Sharpe rather than a win rate, and it does not substitute for the max-statistic bound above. And for a search over hundreds of rules and thousands of cells against roughly three months of data, the minimum backtest length needed before an in-sample optimum can be trusted (Bailey, Borwein, Lopez de Prado and Zhu, 2014) very likely exceeds the sample outright, one more reason to expect a null.

On the program's own numbers the deflation is decisive. Recompute the three-level majority rule in sample and it earns a large Sharpe, about 13 annualized over the 47 in-sample days on the raw signal, about 6 on the levered account. Because the plain majority rule is unregularized, it prints an even larger in-sample Sharpe than the gated method in the companion paper. The expected best-of-search Sharpe under pure noise, though, computed by the deflated Sharpe ratio from the spread of the per-cell Sharpes, is larger still: about 25 annualized at fifty trials, rising to 37 across the roughly 1,800 populated cells. The observed in-sample Sharpe falls below that noise expectation, so the deflated Sharpe ratio, the probability the true Sharpe clears zero once the search is priced in, comes out below 0.001 for any plausible trial count, and the minimum track-record length needed to reach significance is unbounded.

8. Safeguard Five: Guard the Labels and the Clock

Both of the failures in this section come from one root: future information leaking into a pipeline that is meant to be causal. The first is ordinary look-ahead, in labeling or feature construction. The program guards against it by re-implementing the outcome resolver from scratch, with a different algorithm, and checking that the two agree. They agreed on all 64,434 resolutions checked, with entry prices taken only from bars at or before the decision and outcomes only from strictly later bars. Without a check like that, a one-bar leak can manufacture an implausibly high win rate that is pure hindsight.

Leakage can also creep in at the train-test boundary rather than inside a row. Because outcomes are labeled over a window of up to forty-eight hours, a training snapshot taken within two days of the 1 July split has its label resolved partly on prices from after the split, so the training set gets a peek across the boundary. The standard fix is to purge the overlapping-label rows and embargo a band of at least the label horizon around the split, as in purged and embargoed cross-validation (Lopez de Prado, 2018). With roughly hourly scans over twenty coins, that band runs to about a thousand rows. Re-running the split with a 48-hour purge and embargo settles the question directly: the out-of-sample directional win rate of the majority rule moves by less than a tenth of a percentage point, so the null is not an artifact of boundary leakage. The step stays mandatory anyway, because a leak has to be shown immaterial, not assumed to be, and it belongs on the checklist next to the within-row guard.

The second failure is an encoding bug. It is not look-ahead, but it fabricates signal all the same. In the program's own encoder a percentile transform broke ties by the top of the tie block, so a feed sitting at its own modal value got pushed to an extreme code. A binary stress flag that read zero, meaning no stress, on 88.5 percent of rows was encoded as a maximum-conviction short on 91.0 percent of rows, and 98.8 percent of all rows carried at least one of these tie-inflated codes. The fix was a mid-rank percentile, which sends a constant series to the neutral code. The lesson reaches past this one bug: audit the distribution of the encoded signal, not only the raw feature, and treat any code that fires on almost every row as a bug until shown otherwise.

9. Safeguard Six: Defend the Raw Signal, Not the Equity Curve

In the program a five-level signature showed a positive out-of-sample return at the two-to-one bracket, on a taken win rate of 47.4 percent against a 33.3 percent single-trade break-even. That reads like an edge until you look underneath. The raw, pre-sequencing candidate win rate was 32.1 percent, below break-even. The one-position-per-coin rule had quietly collapsed a cluster of losing candidates in one cell, turning a 17.9 percent candidate win rate into a 40.0 percent taken win rate purely by dropping trades. And the profit was concentrated: one large-cap coin, winning four of four, supplied about 61 percent of the net, and dropping the top three coins flipped the whole result negative. So a positive equity curve does not establish a directional edge, because portfolio-construction rules can interact with correlated outcomes to spin returns out of nothing. That concentration is the same correlated-events problem from Safeguard 3 seen in cross-section, a few correlated outcomes standing in for an independent sample.

The decisive test holds the fired events fixed, their coins, their timing, and their resolved outcomes, and shuffles only the trade side, keeping the realized number of long and short trades unchanged so that the strategy's directional tilt, and with it its beta exposure, stays put. Only the assignment of sides moves. That makes the null beta-matched rather than side-balanced. Figure 4 shows the permutation null over two thousand random side assignments. The observed return sits well inside it, at a one-sided permutation probability of about 0.08, with a Monte Carlo standard error near 0.006 at this draw count, short of conventional significance. That is a failure to reject the null of no directional skill, which is not the same as proof that the null holds. Set it beside a candidate win rate below break-even and a profit riding on one coin, though, and the evidence reads as noise. One-position-per-coin can by itself turn random sides into a rising account curve

by collapsing losing clusters, so a positive curve here is entirely consistent with no directional information at all.

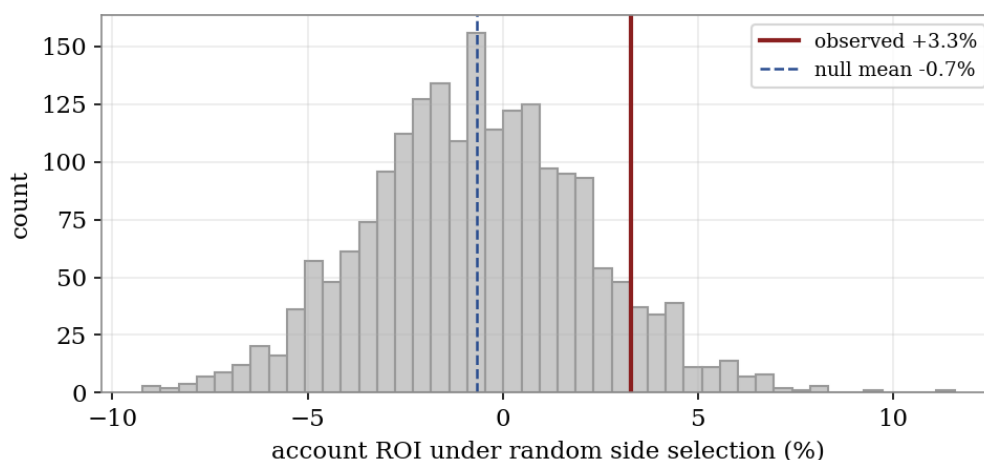


Figure 4. Beta-matched permutation null for the five-level out-of-sample return, randomizing only the trade side over two thousand draws while holding the number of long and short trades fixed. The observed return lies inside the null, at a one-sided permutation probability of about 0.08, not in its tail.

10. Safeguard Seven: Pin Down Implementation Nondeterminism

A language-model strategy in the program first looked mildly profitable, at plus 0.6 percent over the full window. The ninety-five percent capital cap bound often, so on a given timestamp many candidates competed for a limited number of slots, and which of the capped trades got dropped came down to the sort used to break ties. The original code used an unstable sort. Randomize the tie order and the return spreads into the distribution in Figure 5: from a five percent loss to a two percent gain, centered on a mild loss, with the reported plus 0.6 percent sitting in the optimistic tail. The sign of that result was an artifact of sort order, nothing more.

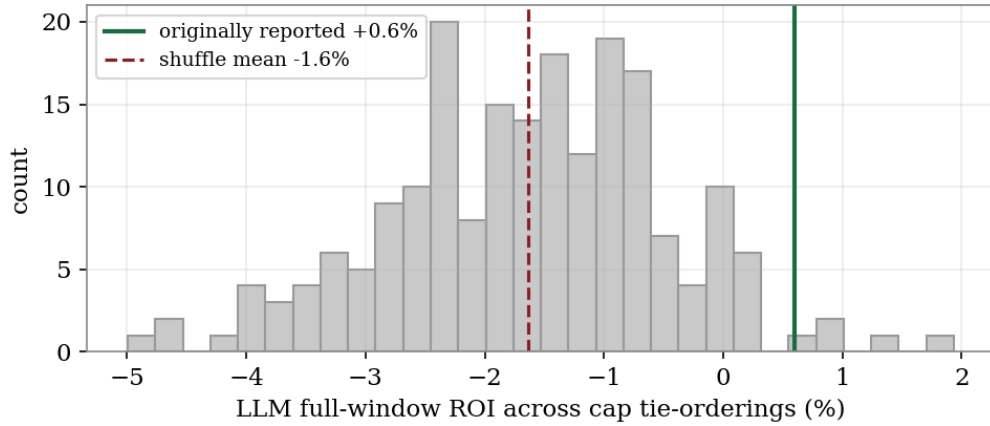


Figure 5. Full-window return of the language-model strategy across random tie-break orderings of a binding ninety-five percent capital cap. The result swings from about a five percent loss to a two percent gain on the tie order alone; its center is a mild loss, so the originally reported plus 0.6 percent is the optimistic tail, not a robust figure.

11. Two Case Studies

The two case studies are worth setting side by side, because each cleared a different subset of the checks and failed one in particular. Table 1 lays them out; the details are already in Safeguards 6 and 7, so I will not repeat them.

Result	First-pass appearance	Check that overturned it	Honest conclusion
Language-model final decision	Plus 0.6% full-window return	Safeguard 7: unstable tie-break under a binding cap	Confirmed artifact; the positive sign was an accident of sort order
Five-level signature	Plus 3.3% out-of-sample at 47.4% taken win rate	Safeguards 6 and 3: candidate win rate 32.1% below the 33.3% break-even; 61% of profit from one coin; side-permutation p about 0.08	Could not be distinguished from chance

Table 1. Two positive results examined by verification, and the specific check that overturned each.

The two near-misses failed at different stages. One was a deterministic artifact that a stability stress caught immediately. The other cleared a first out-of-sample pass and only came apart once the raw signal, the concentration, and the permutation were read together. Neither was a coding mistake in the narrow sense; the resolver and the account arithmetic checked out independently in both. In each case the error was in the inference while the arithmetic held, which is why the

equity curve and the taken win rate misled, and why it took the beta benchmark, the raw signal, the permutation, and the stability stress to see past them.

12. The Protocol as a Checklist

The safeguards fold into a short protocol to run against any candidate edge before you believe it or deploy it.

1. Split chronologically. Fix every parameter on the past and score only on the held-out future. Expect the in-sample number to evaporate, and reserve belief for the test period. Treat one split as the weakest hold-out worth trusting, and prefer combinatorially symmetric cross-validation, reporting a probability of backtest overfitting, when the sample allows.
2. Benchmark against beta. Compare to the one-sided baseline at the exact times and instruments traded, and report the margin over it rather than the raw win rate. Test that margin with the beta-matched permutation of step 6, never on its own.
3. Reduce to independent events. Collapse correlated instruments to one observation per event and block adjacent days before any t-statistic, with a block at least as long as the event-overlap horizon, and prefer cluster-robust or block-bootstrap variance estimates to a bare count of distinct events.
4. Deflate for the search. Divide credit by the number of candidates screened and clear a deflated bar, not the naive one. Use a correction that respects dependence among candidates, a stepwise family-wise procedure or a bootstrap reality check at a stated family-wise alpha (0.05 by convention), and match the tool to the statistic, a deflated Sharpe ratio when the headline is a Sharpe.
5. Guard the labels and the clock. Re-implement the resolver independently, forbid future bars within a row, purge overlapping-label rows and embargo a band of at least the label horizon around the split, and audit the distribution of the encoded signal for codes that fire almost always.
6. Defend the raw signal. Test the selection itself, not the portfolio curve, and prefer a permutation of the thing under study. For a directional result, shuffle only the side while keeping the long-short count fixed, use at least a couple of thousand draws, and judge against the one-sided permutation probability.

7. Pin down nondeterminism. Make ties and other arbitrary choices deterministic, and require a headline result to survive the choices that are not supposed to matter.

Step 5 is a precondition rather than one more step in sequence. A leaked label or a look-ahead feature voids the out-of-sample number outright, so steps 1, 2, and 6 are only ever worth as much as the step 5 beneath them.

13. Discussion and Limitations

The protocol is conservative by design. It exists to reject false positives, and every so often it will also reject a true but weak effect that cannot be separated from noise on the data at hand, as with the equity reversal whose de-collinearized significance lands right at the conventional threshold. That case is worth keeping for the opposite reason to the two crypto near-misses: the protocol does not reject it. The same checks that overturn the artifacts leave a modest true effect standing at the honest bar, which is the evidence that the protocol has real discriminating power rather than a blanket tendency to reject.

That claim about power is testable, so I ran a positive control. When a synthetic directional edge of known strength is planted into a balanced event set with no beta, and the same beta-matched side-permutation test is run, it behaves the way a valid test should. With no edge planted it fires about four percent of the time, close to its nominal size, so it is not rejecting everything in sight. Its power then rises with both the edge and the sample. An eight-point win-rate edge, a 58 percent directional win rate against a 50 percent baseline, is caught about 84 percent of the time at 300 trades and essentially always at 1,000; a five-point edge is caught 93 percent of the time at 1,000. That same eight-point edge, at the 38-trade sample of the five-level case study, is caught only about 17 percent of the time. That is the concrete reason the permutation probability of 0.08 there speaks to limited power at a small sample rather than to the absence of any edge.

Some boundaries. The examples come from one program on particular instruments and windows, so the exact numbers are illustrative, not general; the failure modes are not specific to this program, though, and each maps to a known result in the data-snooping literature. The protocol covers statistical and implementation validity, not economic realism. The account figures already carry a flat 0.1 percent round-trip cost, but slippage, market impact, and capacity are unmodeled, and each is its own necessary check before any deployment. A live program also permits one test

a backtest cannot, reconciling realized live profit and loss against the backtest at matched timestamps; that live-versus-backtest tracking complements the safeguards here and is left to a follow-on paper. And the window is a single, mostly down-trending regime, with a search large relative to three months of data, so by the minimum-backtest-length standard of Safeguard 4 the sample is short. The most useful next step is not more methods but more out-of-sample time across regimes that include a sustained uptrend, ideally under a pre-registered test that fixes the rules before the data arrive.

14. Conclusion

The hardest bias to catch in strategy research is the researcher's own preference for a particular result, and a backtest does nothing on its own to correct it. The seven safeguards here, aimed by the author at the results he most wanted to keep, are what separated the discoveries from the artifacts. Run in full on the program that supplied the examples, they turned two apparent edges into one confirmed artifact, whose positive sign flipped on an arbitrary tie-break, and one result a beta-matched permutation could not tell from chance, which left a null as the honest answer. The checks themselves are not new, and integrated protocols already exist; the contribution is in watching each one applied to a live failure, with the cost of skipping it named. Most candidate edges, examined this way, do not survive. The two in this paper did not.

Author Contributions and Use of AI Tools

The research questions, experimental design, hypotheses, methodological choices, and interpretation of results in this paper are the author's own. All software implementation, including data collection, feature encoding, the backtesting engine, the statistical and permutation tests, the resolvers, and the figures, was written in Python with the assistance of Claude, a large language model by Anthropic, acting as a coding assistant under the author's direction. Claude is a software tool and is not an author of this work. The author is solely responsible for the content, the claims, and any errors.

Data and Code Availability

The analysis scripts behind every table and figure here, the two independent first-touch resolvers, the feature encoder, the account simulator, the search and deflation code, and the side-

randomization permutation test, are written in Python and available from the author on reasonable request, with a companion code repository in preparation. The underlying snapshot dataset and its labeled outcomes are available on the same terms, subject to the terms of the third-party data sources used to build the features. The four robustness analyses reported here, the deflated Sharpe ratio, the purge-and-embargo re-split, the equity block bootstrap, and the side-permutation power control, come from a single deterministic script that reads the snapshot data and the public daily index data and prints every number quoted above. The equity reversal study in Safeguard 3 uses daily index data (S&P 500, Nasdaq Composite, Dow Jones Industrial Average, and Russell 2000) from a public source over 1990 to 2026.

References

- [1] Arnott, R., Harvey, C. R., and Markowitz, H. (2019). A backtesting protocol in the era of machine learning. *Journal of Financial Data Science*, 1(1), 64 to 74.
- [2] Aronson, D. R. (2007). *Evidence-Based Technical Analysis*. Hoboken, NJ: Wiley.
- [3] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., and Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: the effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, 61(5), 458 to 471.
- [4] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., and Zhu, Q. J. (2017). The probability of backtest overfitting. *Journal of Computational Finance*, 20(4), 39 to 69.
- [5] Bailey, D. H., and Lopez de Prado, M. (2014). The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality. *Journal of Portfolio Management*, 40(5), 94 to 107.
- [6] Dashyan, A. (2026). Short-horizon directional non-predictability in cryptocurrency perpetual futures: a multi-method negative result under adversarial verification. Working paper.
- [7] Hansen, P. R. (2005). A test for superior predictive ability. *Journal of Business and Economic Statistics*, 23(4), 365 to 380.
- [8] Harvey, C. R., and Liu, Y. (2015). Backtesting. *Journal of Portfolio Management*, 42(1), 13 to 28.
- [9] Harvey, C. R., Liu, Y., and Zhu, H. (2016). ... and the cross-section of expected returns. *Review of Financial Studies*, 29(1), 5 to 68.
- [10] Ioannidis, J. P. A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8), e124.

- [11] Jegadeesh, N. (1990). Evidence of predictable behavior of security returns. *Journal of Finance*, 45(3), 881 to 898.
- [12] Lehmann, B. N. (1990). Fads, martingales, and market efficiency. *Quarterly Journal of Economics*, 105(1), 1 to 28.
- [13] Lo, A. W., and MacKinlay, A. C. (1990). Data-snooping biases in tests of financial asset pricing models. *Review of Financial Studies*, 3(3), 431 to 467.
- [14] Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.
- [15] Novy-Marx, R. (2016). Backtesting strategies based on multiple signals. Working paper, University of Rochester.
- [16] Politis, D. N., and Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428), 1303 to 1313.
- [17] Romano, J. P., and Wolf, M. (2005). Stepwise multiple testing as formalized data snooping. *Econometrica*, 73(4), 1237 to 1282.
- [18] Sullivan, R., Timmermann, A., and White, H. (1999). Data-snooping, technical trading rule performance, and the bootstrap. *Journal of Finance*, 54(5), 1647 to 1691.
- [19] White, H. (2000). A reality check for data snooping. *Econometrica*, 68(5), 1097 to 1126.